ASTAS System Architecture & Workflow

📋 Table of Contents

1. System Overview
2. Architecture Design
3. Component Roles
4. Data Flow
5. Module Interactions
6. Workflow Explanation

---

1. System Overview

ASTAS (Autonomous Surveillance & Threat Assessment System) is an intelligent surveillance platform that integrates:

- **Computer Vision** for object detection and tracking
- **Multi-Sensor Fusion** (IMU, LiDAR, Audio) for comprehensive situational awareness
- **LLM-Based Reasoning** for intelligent threat assessment
- **Autonomous Control** for camera positioning and response

Key Capabilities

✅ Real-time object detection and classification
✅ Multi-object tracking with behavior analysis
✅ Audio event detection (footsteps, vehicles, alarms)
✅ LiDAR point cloud processing
✅ IMU-based motion analysis
✅ AI-powered threat assessment using LLMs
✅ Autonomous camera control
✅ Alert generation and logging

---

2. Architecture Design

```
┌──────────────────────────────────────────────────────────────┐
│                    ASTAS CORE SYSTEM                         │
```

```
└──────────────────────────────────────────────────────────────┘
                  │
      ┌───────────┴──────────────┐
      │           │              │
      ▼           ▼              ▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│  SENSORS     │ │  COMPUTER    │ │   LLM        │
│              │ │  VISION      │ │  REASONING   │
├──────────────┤ ├──────────────┤ ├──────────────┤
│ • IMU        │ │ • Detection  │ │ • Context    │
│ • LiDAR      │ │ • Tracking   │ │   Building   │
│ • Audio      │ │ • Thermal    │ │ • Threat     │
│ • Camera     │ │ • Behavior   │ │   Assessment │
└──────────────┘ └──────────────┘ └──────────────┘
      │           │              │
      └───────────┴──────────────┘
                  │
                  ▼
            ┌──────────────────┐
            │ DECISION ENGINE  │
            │ & CONTROL        │
            ├──────────────────┤
            │ • Alert Gen      │
            │ • Camera Ctrl    │
            │ • Logging        │
            └──────────────────┘
```

---

3. Component Roles

3.1 Sensor Processing Layer

IMU Processing (`sensors/imu_processing.py`)

Role: Process inertial measurement unit data for motion detection and orientation tracking

Key Functions:

- Kalman filtering for noise reduction
- Orientation estimation (roll, pitch, yaw)
- Vibration detection
- Motion classification (stationary, slow, moderate, rapid)
- Anomaly detection in movement patterns

Input: Raw accelerometer and gyroscope data **Output:** Filtered sensor data, orientation, motion type, vibration alerts

Use Case: Detect unusual vibrations (e.g., fence cutting, explosions), track platform orientation

---

LiDAR Processing (`sensors/lidar_processing.py`)

Role: Process 3D point cloud data for obstacle detection and environment mapping

Key Functions:

- Voxel downsampling for efficiency
- Ground plane removal using RANSAC
- Euclidean clustering for object segmentation
- Object classification (person, vehicle, unknown)
- Occupancy grid generation

Input: 3D point cloud (x, y, z coordinates) **Output:** Detected objects with positions, bounding boxes, classifications

Use Case: Detect objects in low-visibility conditions, measure distances accurately

---

Audio Signal Analysis (`sensors/audio_signal_analysis.py`)

Role: Analyze audio signals for acoustic event detection

Key Functions:

- FFT-based frequency analysis
- Spectrogram computation
- Event signature matching (footsteps, vehicles, gunshots, alarms)
- Sound source direction estimation (TDOA)
- Temporal pattern analysis (rhythm detection)

Input: Raw audio signal **Output:** Detected acoustic events with confidence, frequency ranges, intensity

Use Case: Detect approaching vehicles, identify alarms, recognize breaking glass

---

3.2 Computer Vision Layer

Object Detection (`cv/object_detection.py`)

Role: Detect and classify objects in video frames using YOLOv8

Key Functions:

- Real-time object detection
- Multi-class recognition (person, vehicle, animal, etc.)
- Confidence filtering
- Zone-based filtering
- Performance tracking (FPS, latency)

Input: RGB video frame **Output:** List of detections with bounding boxes, classes, confidence scores

Use Case: Identify persons, vehicles, animals in surveillance footage

---

Motion Tracking (`cv/motion_tracking.py`)

Role: Track detected objects across frames and analyze behavior

Key Functions:

- Multi-object tracking (DeepSORT-inspired)
- Track ID assignment and maintenance
- Trajectory recording
- Behavior analysis (loitering, rapid movement, direction changes)
- IoU-based data association

Input: Detections from object detector, video frame **Output:** Tracked objects with IDs, trajectories, velocities, behaviors

Use Case: Monitor continuous movement patterns, detect loitering behavior

---

Thermal Analysis (`cv/thermal_analysis.py`)

Role: Process thermal/infrared imagery for heat signature detection

Key Functions:

- Heat signature detection

- Temperature estimation
- Signature classification (person, vehicle, animal)
- Thermal image enhancement (CLAHE)
- RGB-thermal fusion

Input: Thermal/infrared image **Output:** Heat signatures with estimated temperatures, classifications

Use Case: Night vision, detect hidden persons, identify vehicle engines

---

3.3 LLM Reasoning Layer

Decision Engine (`llm/inference/decision_engine.py`)

Role: Assess threat levels using AI reasoning

Key Functions:

- Context aggregation from all sensors
- LLM-based threat assessment (if model available)
- Rule-based fallback system
- Threat level classification (low, medium, high, critical)
- Reasoning explanation generation
- Action recommendation

Input: Aggregated context (detections, behaviors, sensor data, zone info) **Output:** Threat assessment with level, score, reasoning, recommended actions

Threat Assessment Logic:

```
Factors Considered:
- Zone type (restricted, caution, safe)
- Time of day (night = higher risk)
- Object types and count
- Behavior patterns (loitering, rapid movement)
- Audio events (gunshots, alarms)
- Historical patterns
- Sensor anomalies

Scoring:
0.0-0.4  → LOW
0.4-0.65 → MEDIUM
0.65-0.85 → HIGH
0.85-1.0 → CRITICAL
```

3.4 Control System Layer

PID Controller (`control_system/pid_controller.py`)

Role: Implement proportional-integral-derivative control for smooth system responses

Key Components:

CameraController:

- Tracks moving targets
- Smooth pan/tilt control
- Velocity-based control with acceleration limits

TrajectoryPlanner:

- Path planning through waypoints
- Velocity profile computation
- Acceleration-constrained following

Input: Target position, current state **Output:** Control commands (velocities, positions)

Use Case: Smooth camera tracking of suspects, autonomous patrol routes

---

3.5 Simulation Layer

Environment Simulator (`simulation/env_sim.py`)

Role: Generate realistic surveillance scenarios for testing

Key Functions:

- Multi-agent simulation (persons, vehicles, animals)
- Behavior patterns (patrol, loiter, rush, perimeter breach)
- Restricted zone definition
- Event generation
- Realistic rendering

Agent Behaviors:

- **PATROL:** Follow predefined waypoints
- **LOITER:** Slow circular movement in area
- **RUSH:** Fast straight-line movement
- **RANDOM_WALK:** Unpredictable wandering
- **PERIMETER_BREACH:** Intentional zone violation

---

4. Data Flow

Complete Pipeline (Frame-by-Frame)

```
Step 1: SENSOR DATA ACQUISITION
┌──────────────────────────────────────┐
│ Camera Frame (1920x1080 RGB)         │
│ IMU Data (accel, gyro)               │
│ LiDAR Point Cloud (N x 3)            │
│ Audio Signal (44.1kHz)               │
└──────────────────────────────────────┘
              ↓
Step 2: SENSOR PROCESSING
┌──────────────────────────────────────┐
│ IMU → Orientation, Vibration         │
│ LiDAR → Objects, Distances           │
│ Audio → Events (footsteps, etc.)     │
└──────────────────────────────────────┘
```

```
                ↓
Step 3: COMPUTER VISION
┌──────────────────────────────────────────┐
│ YOLOv8 Detection →                       │
│   [person: 0.95, car: 0.87, ...]         │
└──────────────────────────────────────────┘
                ↓
Step 4: MULTI-OBJECT TRACKING
┌──────────────────────────────────────────┐
│ Track Matching → ID Assignment →         │
│ Trajectory Update → Behavior             │
│   [ID=1: loitering=True, ...]            │
└──────────────────────────────────────────┘
                ↓
Step 5: CONTEXT BUILDING
┌──────────────────────────────────────────┐
│ {                                        │
│   detections: [person, car],             │
│   zone: "restricted",                    │
│   time: "night",                         │
│   loitering: True,                       │
│   audio_events: [footsteps],             │
│   vibration: False                       │
│ }                                        │
└──────────────────────────────────────────┘
                ↓
Step 6: LLM THREAT ASSESSMENT
┌──────────────────────────────────────────┐
│ Prompt → LLM/Rules → Assessment          │
│                                          │
│ Output:                                  │
│   threat_level: "HIGH"                   │
│   score: 0.78                            │
│   reasoning: "Person loitering in        │
│               restricted zone..."        │
│   actions: ["Alert operator",            │
│             "Track continuously"]        │
└──────────────────────────────────────────┘
                ↓
Step 7: DECISION & CONTROL
┌──────────────────────────────────────────┐
│ IF threat >= HIGH:                       │
│   - Generate alert                       │
│   - Log event                            │
```

```
│  - Track with camera          │
│  - Notify operator            │
└──────────────────────────────────────────┘
                ↓
Step 8: OUTPUT
┌──────────────────────────────────────────┐
│ • Annotated video frame          │
│ • Alert messages                 │
│ • Camera control commands        │
│ • Event logs                     │
└──────────────────────────────────────────┘
```

---

5. Module Interactions

Interaction Diagram

```
main.py (ASTASCore)
    │
    ├──> object_detection.py
    │        └──> Returns: List[Detection]
    │
    ├──> motion_tracking.py
    │        └──> Input: Detections
    │        └──> Returns: List[Track] with behaviors
    │
    ├──> imu_processing.py
    │        └──> Returns: IMU data, vibration status
    │
    ├──> lidar_processing.py
    │        └──> Returns: 3D objects, distances
    │
    ├──> audio_signal_analysis.py
    │        └──> Returns: Audio events
    │
    ├──> _build_threat_context()
    │        └──> Aggregates all data → Context Dict
    │
    ├──> decision_engine.py
    │        └──> Input: Context
    │        └──> Returns: ThreatAssessment
    │
```

```
└──> pid_controller.py (CameraController)
        └──> Input: Target position
        └──> Returns: Pan/tilt commands
```

Data Dependencies

```
Detection → Tracking → Behavior Analysis
   ↓        ↓        ↓
   └─────────────────┴─────────────────┴──→ Context Building
                     ↓
Sensors (IMU, LiDAR, Audio) ──────────────┘
                     ↓
                Threat Assessment
                     ↓
               Decision & Control
```

---

6. Workflow Explanation

6.1 Startup Sequence

1. **Configuration Loading**
   - Load `config/config.yaml`
   - Set detection thresholds, sensor parameters, LLM settings
2. **Module Initialization**
   - Initialize object detector (load YOLO model)
   - Initialize tracker (set parameters)
   - Initialize sensor processors
   - Initialize LLM engine (load model or use rules)
   - Initialize camera controller
3. **Ready State**
   - System ready to process frames
   - All modules loaded and configured

6.2 Per-Frame Processing Loop

```python
```

```
for each frame:
    # 1. Capture data
    frame = get_camera_frame()
    sensor_data = get_sensor_readings()

    # 2. Detect objects
    detections = detector.detect(frame)

    # 3. Track objects
    tracks = tracker.update(detections)

    # 4. Analyze behaviors
    for track in tracks:
        behavior = analyze_behavior(track)

    # 5. Process sensors
    imu_result = imu_processor.process(sensor_data['imu'])
    lidar_objects = lidar_processor.detect(sensor_data['lidar'])
    audio_events = audio_processor.detect(sensor_data['audio'])

    # 6. Build context
    context = aggregate_data(
        detections, tracks, behaviors,
        imu_result, lidar_objects, audio_events
    )

    # 7. Assess threat
    assessment = llm_engine.assess_threat(context)

    # 8. Make decisions
    if assessment.threat_level in ['high', 'critical']:
        generate_alert(assessment)
        track_with_camera(primary_target)
        log_event(assessment)

    # 9. Update display
    render_output(frame, tracks, assessment)
```

6.3 Threat Assessment Flow

```
Context Input
    ↓
┌──────────────────────────────────────────────┐
```

```
│ Rule-Based Scoring:          │
│                              │
│ Base Score = 0.0             │
│                              │
│ + Restricted Zone: +0.3      │
│ + Night Time: +0.2           │
│ + Loitering: +0.25           │
│ + Multiple Persons: +0.2     │
│ + Audio Event (gunshot): =1.0 │
│ + Vibration: +0.1            │
│ + Unusual Pattern: +0.15     │
│                              │
│ Total Score = SUM (max 1.0)  │
└──────────────────────────────┘
   ↓
┌──────────────────────────────┐
│ Classify Threat Level:       │
│                              │
│ < 0.4  → LOW                 │
│ < 0.65 → MEDIUM              │
│ < 0.85 → HIGH                │
│ ≥ 0.85 → CRITICAL            │
└──────────────────────────────┘
   ↓
┌──────────────────────────────┐
│ Generate Actions:            │
│                              │
│ LOW: Continue monitoring     │
│ MED: Increase surveillance   │
│ HIGH: Alert operator, track  │
│ CRIT: Immediate alert, lockdown│
└──────────────────────────────┘
```

6.4 Camera Control Flow

```
Target Detected
   ↓
Calculate Error
(target_pos - frame_center)
   ↓
Normalize Error (-1 to 1)
   ↓
Apply Smoothing Filter
```

```
    ↓
PID Controller
(Proportional + Integral + Derivative)
    ↓
Generate Control Output
(pan_speed, tilt_speed)
    ↓
Apply Rate Limits
(max 30°/second)
    ↓
Update Camera Position
(new_pan, new_tilt)
    ↓
Send Commands to Hardware
```

---

7. System Timing

Real-Time Performance Targets

Module	Target Latency	Actual (typical)
Object Detection	< 50ms	30-40ms (GPU)
Tracking	< 10ms	5-8ms
Sensor Processing	< 5ms	2-4ms
LLM Assessment	< 200ms	150-300ms
Total Pipeline	**< 300ms**	**200-350ms**

Frame Rate: 30 FPS (33ms per frame)

- Detection runs every frame
- LLM assessment runs when threat detected or every 10 frames

---

8. Example Scenario Walkthrough

Scenario: Unauthorized Person in Restricted Zone at Night

Frame 1-10: System idle, monitoring

- Detection: None
- Threat: LOW

Frame 11: Person detected

- Detection: [person: 0.92]
- Track: New track ID=1
- Threat: LOW (normal movement)

Frame 11-30: Person approaching restricted zone

- Track: ID=1 moving toward red zone
- Behavior: Normal speed
- Threat: MEDIUM (person in caution zone)

Frame 31: Person enters restricted zone

- Zone violation detected
- Time: Night (multiplier +0.2)
- Threat: HIGH (score=0.72)
- Action: Alert generated, camera tracking initiated

Frame 32-60: Person loitering

- Loitering detected (>2 seconds stationary)
- Threat: HIGH (score=0.85)
- Audio: Footsteps detected
- Action: Continuous tracking, operator notified

Frame 61: Breaking glass sound

- Audio event: Breaking glass
- Threat: CRITICAL (score=0.95)
- Action: IMMEDIATE ALERT, log event, possible alarm trigger

This complete flow demonstrates how ASTAS processes multi-modal data to make intelligent security decisions.

End of Documentation